

Vector Database - A Deep Dive

#141: A Beginner's Guide to the AI Stack's Most Misunderstood Component



MAXINE MEURER AND NEO KIM

APR 25, 2026  PAID

Get my system design playbook for FREE on
newsletter signup:

✓ Subscribed

- *Share this post & I'll send you some rewards for the referrals.*

You've built a search feature.

Then a user types a query, your application hits the database, and results come back ranked by relevance.

Simple enough, it works perfectly.

But you thought about how search needs to actually understand what users meant, not just match the exact words they typed.

Exact match vs Meaning match

Traditional DB Search

User Query: "log in"

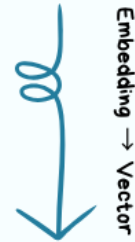


DB:
[login page]
[log in help]
[reset password]

Exact Matches Only

Semantic Search

User Query: "how do I
log in"



Matches:
[account access]
[sign in help]
[password recovery]

Meaning Based Matches

A user searching for "how do I log in" should find results about "account access" and "sign in" even if those exact words never appear in their query.

This kind of search, where the database understands meaning rather than just matching words, is called **semantic search**. It requires a completely different approach to how you store and search your data.

Yet if you try adding this in as a feature, everything that made your traditional relational database fast will stop working.

The tools you relied on: indexes, WHERE clauses, keyword matching were all built for finding exact values.

Semantic search does NOT work with exact values.

Instead, it works with meaning, and meaning is harder to search efficiently. A search that took milliseconds before now takes seconds, and the bigger your dataset, the worse it gets.

Your database is NOT broken.

It's doing exactly what you told it to do: comparing your search against every single item in your database, one by one, because there is no other way to find "similar" when there is NO exact match.

This is where vector databases come in.

Not because they are trendy, but because traditional databases were never designed to handle this kind of search at scale.

If you're a software engineer encountering the term "vector database" for the first time, whether your team is adding AI features or you keep seeing it mentioned everywhere and want to understand what it actually means, this newsletter explains it from the ground up...

[Webinar] Stop babysitting your coding agents (Partner)

WEBINAR

How to stop babysitting your agents

MAY 6, 2026 | 9 AM PT | 12 PM ET

Register Now



Agents can generate code. Getting it right for your system, team conventions, and past decisions is the hard part – you end up wasting time and tokens in correction loops.

More MCPs give agents access to information, but not understanding. The teams pulling ahead use a context engine to give agents exactly what they need.

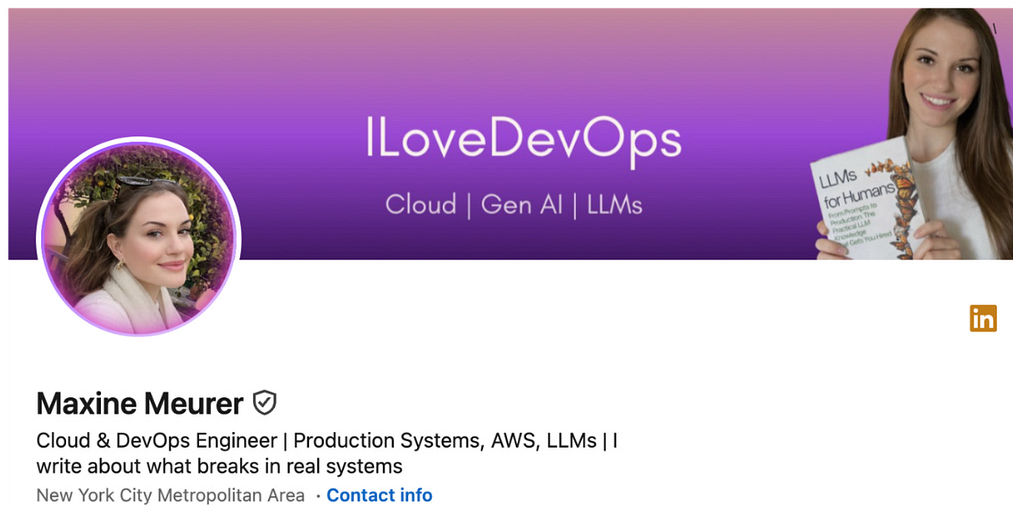
[Join Unblocked live on May 6 \(FREE\)](#) to see:

- Where teams get stuck on the AI maturity curve
- How a context engine solves for quality, efficiency, and cost
- Live demo: the same coding task with and without a context engine

Register Now

(Thanks to [Unblocked](#) for partnering on this post.)

I want to introduce **Maxine** as the guest author.



She's a cloud infrastructure engineer who spends her days scaling databases, debugging production incidents at 2 am, and writing about what actually works in production.

You can get a copy of her **LLMs for Humans: From Prompts to Production (at 30% off right now)**. It's 20 chapters of practical applied AI with real production context, not theory. And it'll help you get smarter about using AI tools in infrastructure workflows.

Checkout her work:

- [Substack - I Love DevOps](#)
- [LinkedIn](#)
- [Threads](#)

Plus, if you're thinking about making a career move into cloud or DevOps and want a structured path to get there, get a copy of her **The DevOps Career Switch Blueprint**.

Inside this newsletter, you'll get:

- **Why traditional search breaks.** How exact-match databases fail with semantic meaning and why $O(n)$ similarity search doesn't scale.
- **How vector search works.** Embeddings, high-dimensional space, and why similarity is measured by distance instead of exact matches.
- **The core architecture.** Storage, indexing (HNSW), and query layers, with key tradeoffs in cost, latency, and accuracy.
- **Query execution in practice.** ANN search, metadata filtering (pre vs post), and ranking, including how real systems return results.
- **Production realities.** Scaling, replication, cost structure, monitoring, and failure modes teams hit in production.
- **Decision framework.** When pgvector is enough, when to use a dedicated vector DB, and when distributed systems become necessary.

Golden members get all posts like these!...

✓ Subscribed

Why Traditional Databases Struggle with Embeddings

Traditional relational databases are good at exact matches and range queries.

They use indexes and data structures so the database can jump directly to relevant rows without scanning the entire table.

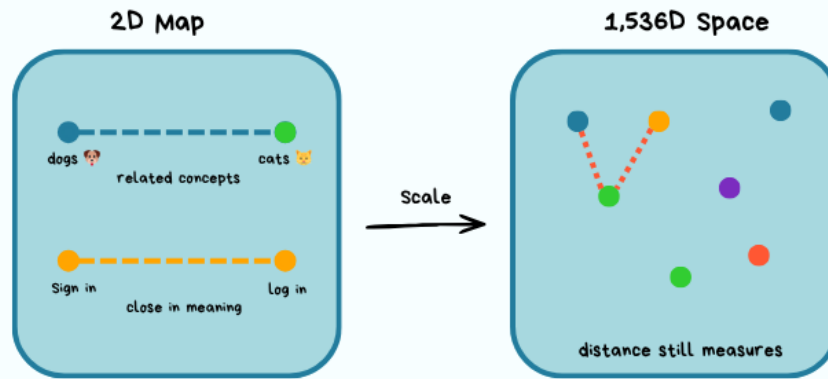
When you query `WHERE user_id = 12345`, database uses an index to find that row instantly, the same way you'd look up a word in a dictionary: you don't read every page, you jump straight to the right section. Also when you query, `WHERE created_at > '2026-01-01'`, it works the same way for dates.

But embeddings break this model because there is no exact value to look up!

An **embedding** is a list of numbers that represents the meaning of a text.

Words, sentences, or entire paragraphs get converted into these numbers by an AI model, and *similar meanings produce similar numbers*.

You can't see it, but distance still works!



Each word or concept becomes a list of numbers. Similar meanings → numbers that are close together

"Log in" and "sign in" are 4 characters apart in text. In vector space, they're almost the same point.

Instead of finding a specific row, you find rows closer in meaning, and "close" requires measuring distance in high-dimensional space rather than matching values. Think of it like a regular map with latitude and longitude, except you're working with 1,536 dimensions instead of just 2.

Let's walk through a concrete example to see why this creates a problem...

Imagine you have a document database where each document gets split into chunks.

And think of chunks like cutting a book into individual paragraphs so you can search each one separately. Each chunk gets converted into a 1,536-dimensional vector using an embedding model like OpenAI's text-embedding-3-small:

1. A user submits a search query
2. App converts the query into a vector using the same embedding model
3. *Now you have to find the "most similar" document chunks*

Without specialized indexing, the only way to find "most similar" ones is to calculate the distance between your query vector and every chunk vector in your database, sort by distance, and return the top results.

Let's break down what it means...

A vector is simply a list of numbers. In AI, those numbers represent meaning.

When you convert text into an embedding, you're assigning it a location in a very high-dimensional space (imagine a map with 1,536 dimensions instead of the normal 2, latitude and longitude). Similar pieces of text end up close to each other on this map, while unrelated text ends up far apart.

When a user searches for something, their query also gets converted into coordinates on this map. To find the most relevant results, you need to measure the distance from the query's location to every document chunk's location in your database, then return the closest ones.

But here's the problem...

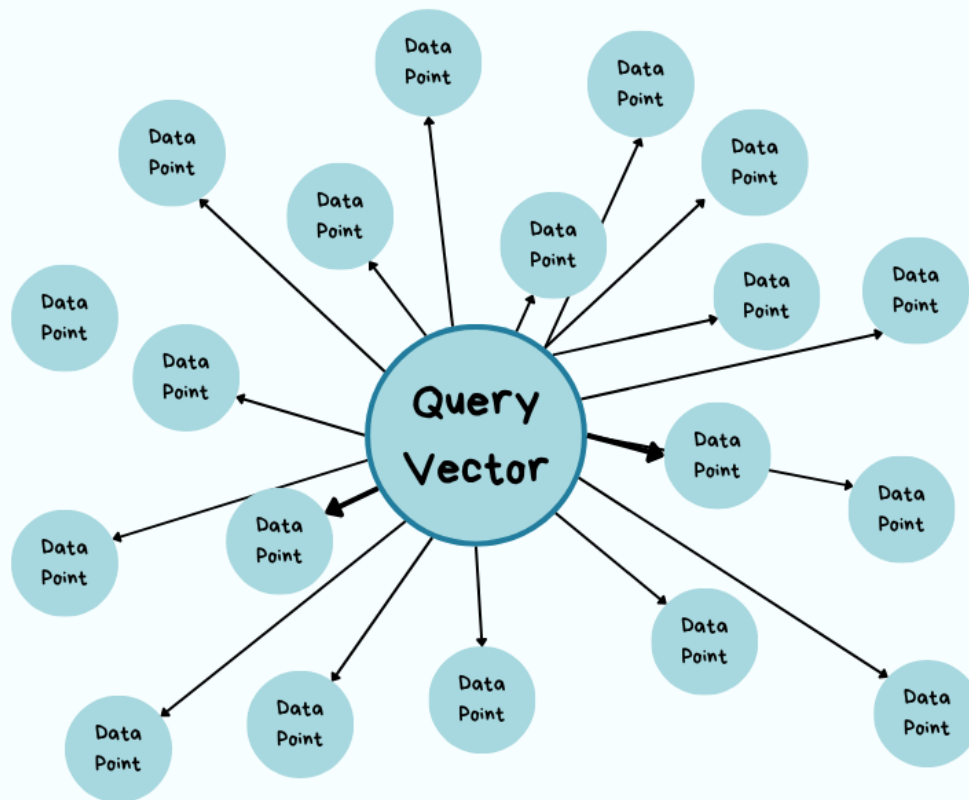
Without an index, there's no shortcut. You must measure the distance to each item.

Now imagine you're standing in Times Square in New York City, and you need to find the 10 closest coffee shops. Without a map or any organized system, your only option is to:

1. Walk to every building
2. Check if it's a coffee shop
3. Measure how far it is from Times Square
4. Keep a list of the 10 closest ones you've found

If there are 10k buildings, you check all 10k. And if there are 10 million buildings, you check all 10 million.

Without an index, you compare against EVERYTHING



10k queries → fast

1M queries → slow

10M queries → dead

The more buildings that exist, the longer it takes...

Put simply, it takes **$O(n)$** time, and it grows linearly with the size of your dataset.

In practical terms, if you have 10,000 document chunks and each distance calculation takes 0.0001 seconds, one query takes 1 second. That's not terrible.

But what if you have 10 million chunks?

That same query now takes 1,000 seconds, i.e., 16 minutes. And if your application serves 100 queries per second from different users? Your system immediately grinds to a halt!

This is why traditional databases struggle with embeddings...

Their indexes are built for exact matches.

They can jump straight to `user_id = 12345` without checking every row. But with vectors, there are no exact matches, only “similar” matches, and similarity requires measuring distances.

Let's now imagine finding a similar song to the one you already like...

You cannot just look it up by name; you have to compare it against every other song across hundreds of characteristics: tempo, key, energy, mood, and so on. The more songs in your library, the longer it takes.

Vector search works the same way, except instead of songs, you compare text, and instead of a few characteristics, you are comparing across 1,536 dimensions at the same time.

Without specialized techniques to avoid checking everything, the math simply doesn't scale.

This is where people often bring up [pgvector](#), a PostgreSQL extension that adds vector search capabilities to Postgres. It's a good option for smaller datasets and lets you avoid adding another database to your stack. This keeps operational efforts very low.

For datasets with fewer than 100,000 vectors and *moderate* query volumes, pgvector works well and keeps your infrastructure simpler.

But as your dataset grows into the millions and your query latency requirements tighten, you hit the limits of what a general-purpose

database can do for this specific workload.

You don't need a specialized vector database just because you're working with embeddings, but you need to understand when the operational trade-offs tip in favor of dedicated infrastructure.

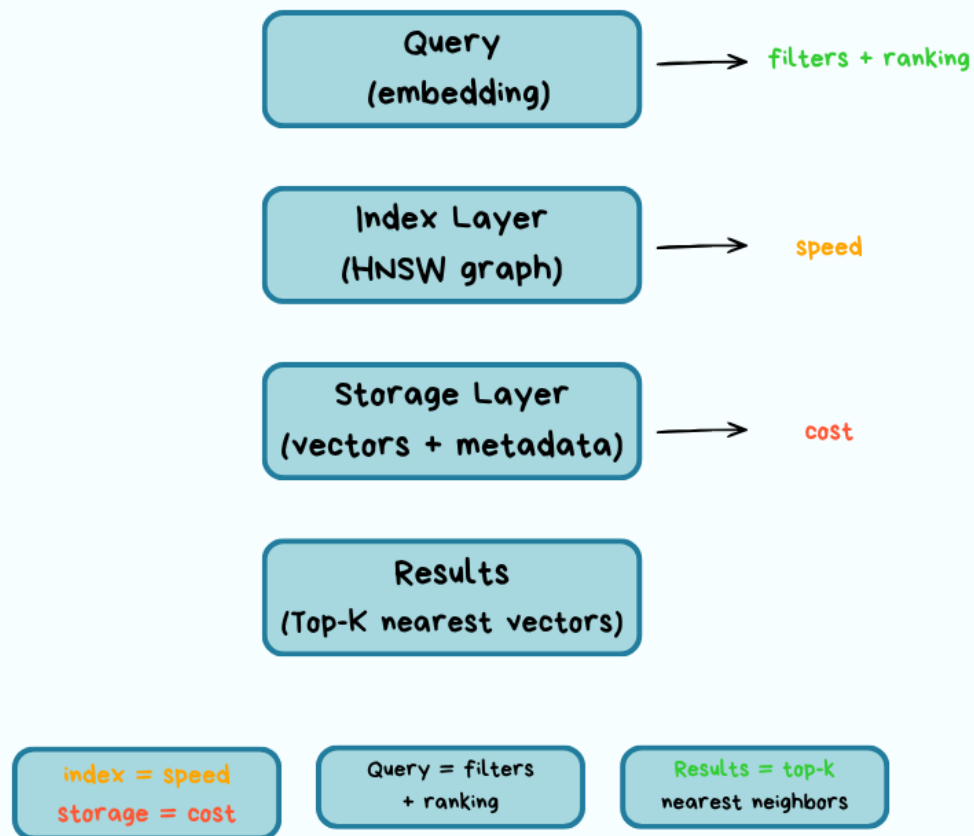
This brings us to the core challenge that vector databases solve...

How do you find approximate nearest neighbors in high-dimensional space without comparing against every vector, while still returning results that are "good enough" in terms of accuracy?

The answer involves building specialized index structures that trade accuracy for massive speed improvements, and understanding those trade-offs is essential for operating these systems in production.

Architecture: How It Works Under the Hood

Vector DB: 3-Layer Architecture



Vector databases have three key layers: *storage*, *index*, and *query*.

Each layer involves trade-offs that directly affect your operational concerns: cost, latency, accuracy, and scalability.

Let's break down what matters from an infrastructure perspective...

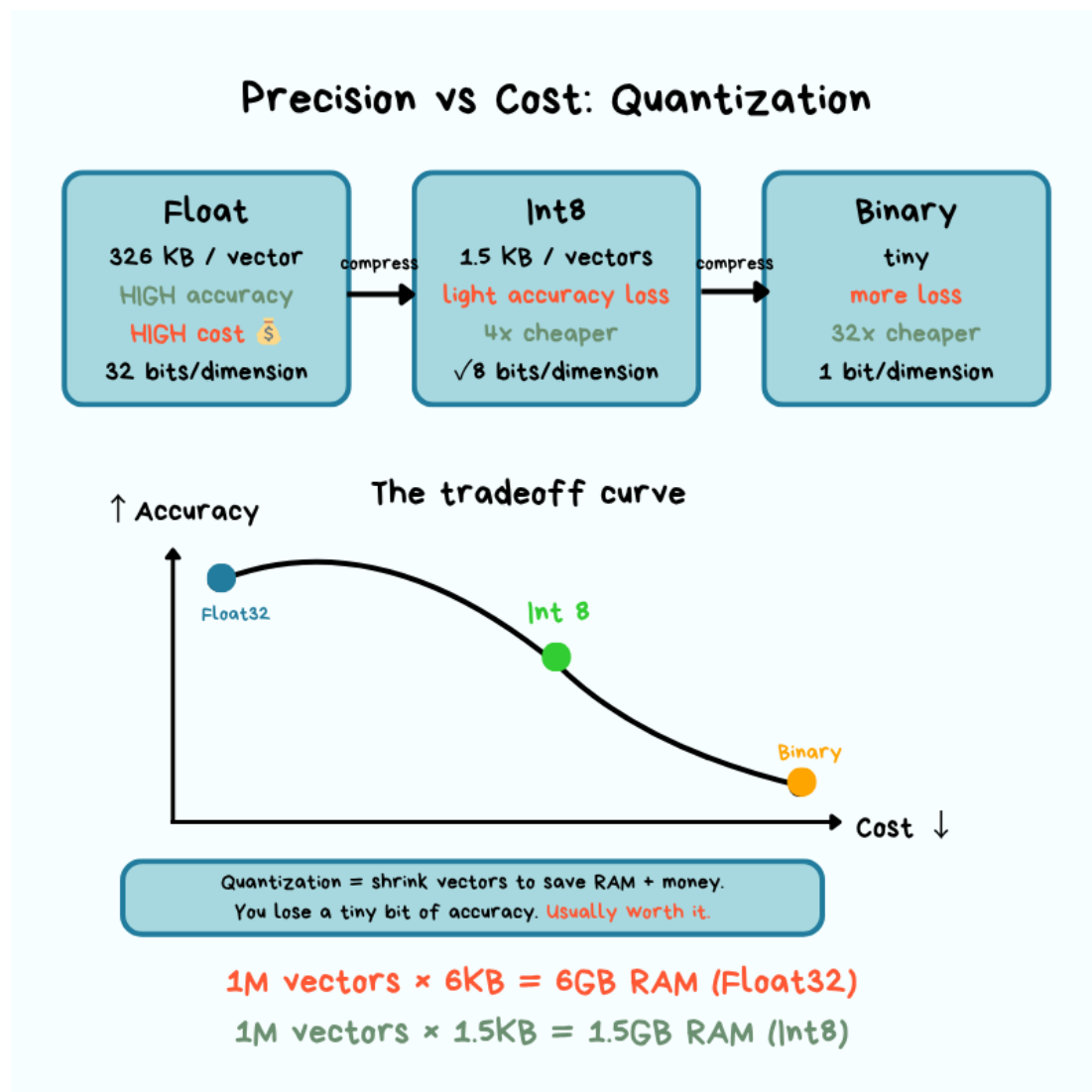
Storage Layer: Where Vectors Live

Vectors are massive.

A single 1,536-dimensional vector using 32-bit floats takes up 6KB of storage. If you multiply that across millions of vectors, storage costs add up fast. This is a problem every vector database must solve at its foundation: *the storage layer determines how vectors are physically stored, compressed, and retrieved from disk*. This layer focuses on compression techniques that reduce both storage costs and the memory footprint while preserving as much accuracy as possible.

The most common approach is quantization.

Quantization reduces the precision of each dimension to use fewer bits.



Instead of storing each dimension as a 32-bit float, you quantize to 8-bit integers (scalar quantization) or binary representations (binary quantization).

This can reduce storage and memory requirements by 4x or more, but it introduces approximation error. **Approximation error** is the gap between the compressed version of data and the original.

When you round numbers to save space, your calculations become slightly less precise. In vector search, this means you might occasionally miss a result that was truly the closest match, but in practice, the difference is small enough that most applications never notice it.

In production, you're constantly trading off storage costs for search accuracy, and the right balance depends on your use case.

Another critical decision is the trade-off between memory and disk...

For **maximum query speed**, you want your vector index in memory, but that's expensive at scale. Some vector databases support tiered storage, where frequently accessed vectors stay in memory while cold data lives on disk, or hybrid approaches in which the index structure stays in memory while raw vectors get pulled from disk when needed.

These architectural choices directly affect your cost structure and query latency percentiles.

From an operational perspective, you need to understand how your chosen vector database handles durability and recovery:

- *What happens if a node crashes mid-write?*
- *How long does it take to rebuild indexes after a restart?*
- *Are vectors written to disk synchronously or asynchronously?*

These aren't theoretical questions. They're the difference between acceptable downtime and a major incident when something goes wrong.

Indexing Layer: Graph Structure

This is where the “magic” happens, and by magic I mean carefully engineered data structures that let you find approximate neighbors without exhaustive search.

The most popular algorithm in production vector databases is **HNSW** (Hierarchical Navigable Small World), and it's worth understanding at a conceptual level even if you're not implementing it yourself.

The core idea behind HNSW is based on “small world” networks, the same principle that says you're only a few connections away from any other person on the planet.

Instead of comparing your query vector against every vector in your database, HNSW builds a multi-layer graph in which similar vectors are connected to each other. When you query, the algorithm starts at the top layer and navigates toward your target by following edges to similar neighbors, dropping through layers until it reaches the most similar vectors at the bottom layer.

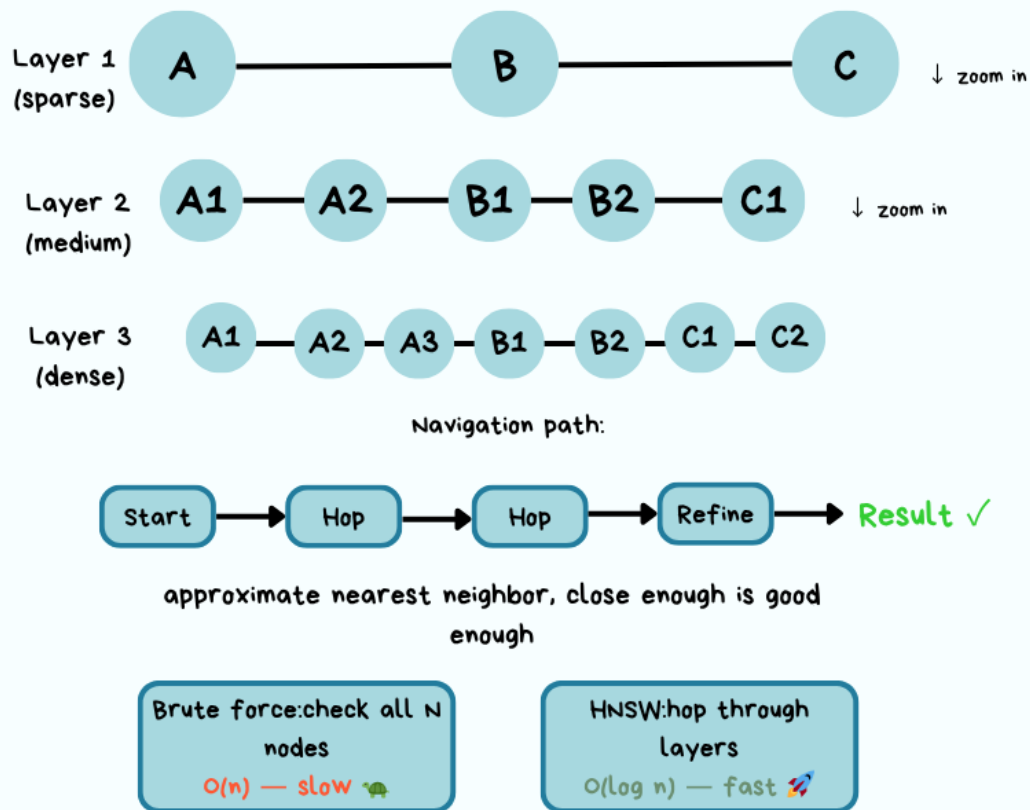
Remember our map analogy?

Your vectors are locations on a high-dimensional map, and you're trying to find the closest ones to your query without checking every single location. HNSW solves this by building a navigation system with different zoom levels.

Think of it like Google Maps navigating from New York to a specific coffee shop in San Francisco:

- **Top layer (zoomed out):** From New York, HNSW identifies the general direction of your target, the West Coast, and follows connections to get you to California. You're not considering every city in America, just the major waypoints that get you closer to where you need to be.
- **Middle layers (zooming in):** Now you're looking at California. The algorithm narrows down to the San Francisco Bay Area, following connections between nearby locations. You're still not checking every street, just the neighborhoods that are directionally correct toward the coffee shop you want to visit.
- **Bottom layer (street level):** Finally, you're at the specific neighborhood and can see individual addresses. The algorithm follows the last few connections to find the exact coffee shop location (or, in vector terms, the most similar document chunks).

HNSW: You don't search everything you **navigate**,



At each zoom level, you're following connections between “nearby” locations without checking everywhere.

You only need to follow the connections that get you closer to your target. This is how HNSW turns an $O(n)$ problem into something much faster: *it builds a navigation structure that lets you skip large portions of the map in the wrong direction.*

The key trade-off in HNSW is speed versus accuracy, controlled by two parameters: the number of connections each vector has (called “M” or

“max connections”) and the number of neighbors to check during search (called “ef” or “exploration factor”).

Higher values mean better accuracy, but slower queries and larger indexes...

Lower values mean faster queries, but you might miss some relevant results. In production, you tune these parameters based on your latency requirements and tolerance for accuracy.

Building the HNSW index is expensive.

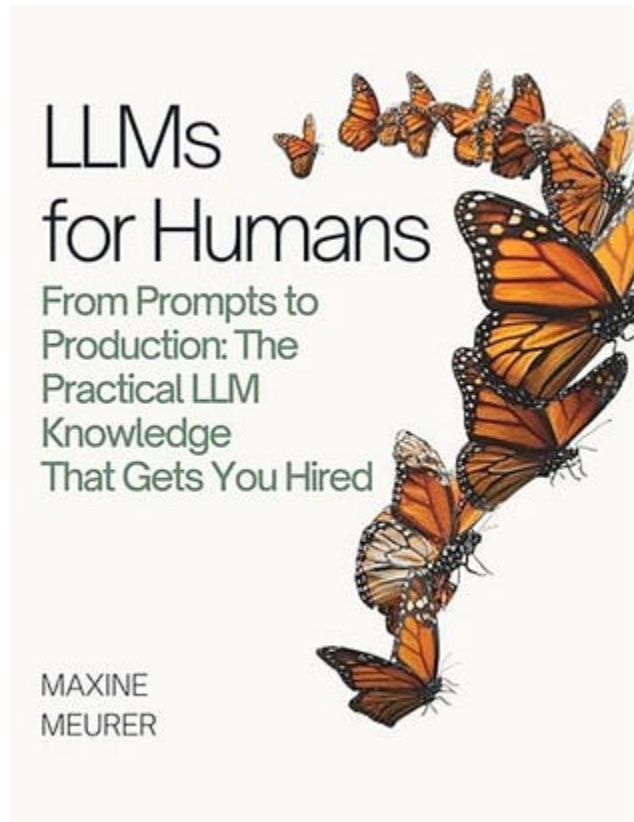
Inserting vectors requires calculating distances and updating graph connections, which is why bulk index building often happens offline or during low-traffic periods. Index build time matters because it affects how quickly you can recover from failures, deploy updates, or onboard new data.

You do not need to understand the math behind HNSW to work with vector databases.

Just like driving a car, you might not know how the engine works, and that doesn't affect how you drive. But knowing that the engine exists and what it does helps you figure out what is wrong when the car breaks down.

What you need to know is that HNSW produces approximate results, not perfect ones, and it's possible to adjust its approximation. When search results seem wrong, queries suddenly slow down, or the index fails to build, this understanding helps you identify the problem.

Get a copy of **LLMs for Humans: From Prompts to Production** right now and save 30%.



Reminder: this is a teaser of the subscriber-only newsletter, exclusive to my golden members.

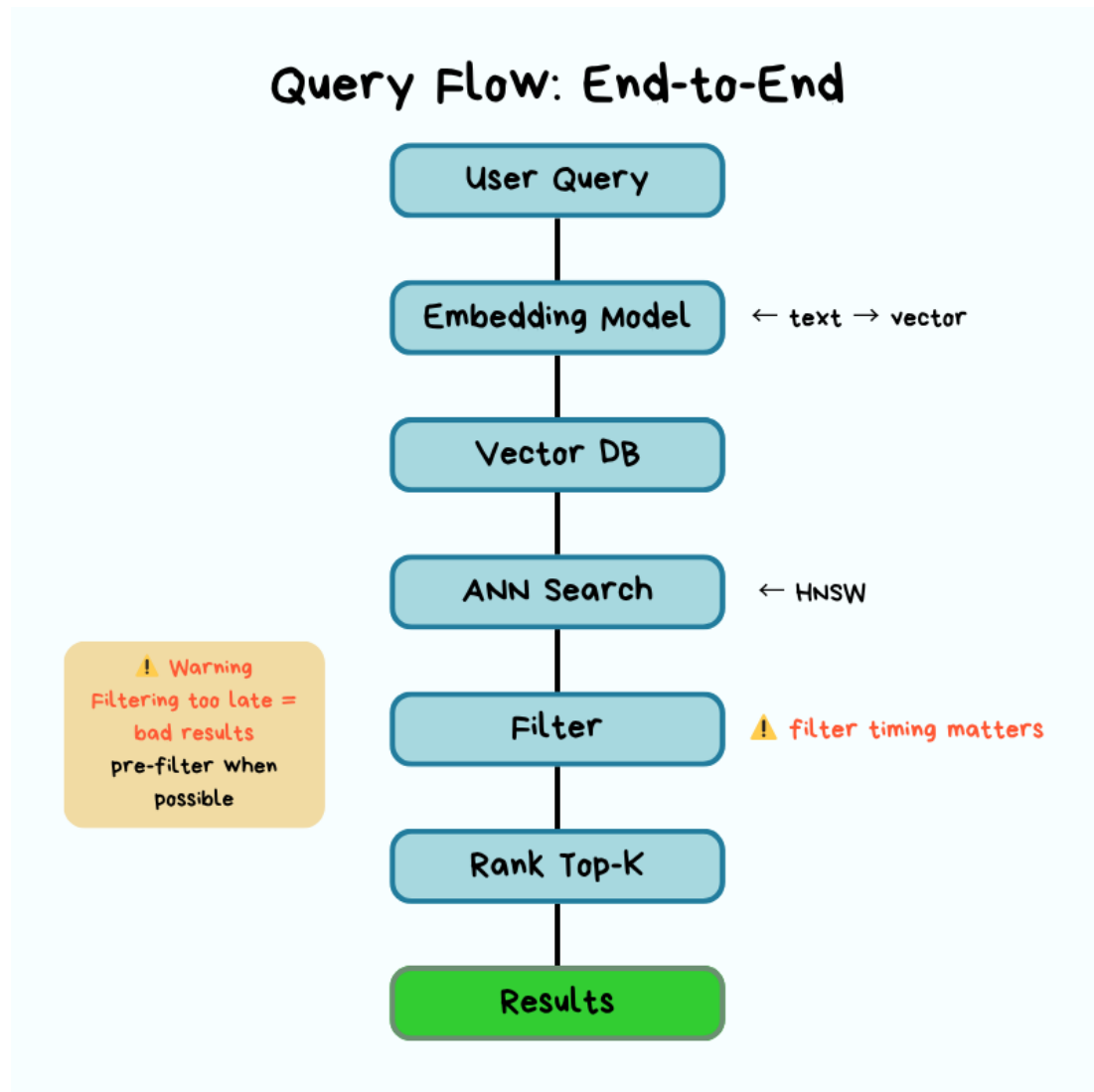
When you upgrade, you'll get:

- **High-level architecture of real-world systems.**
- Deep dive into how popular real-world systems work.
- **How real-world systems handle scale, reliability, and performance.**

And much more!

[Unlock Full Access](#)

Query Layer: How Requests Move Through System



When a query hits your vector database, it follows a specific path:

1. Query vector enters the system
2. Indexing layer finds approximate nearest neighbors
3. Metadata filters get applied
4. Results get ranked
5. Then top-k matches get returned

Each step has operational implications...

Metadata filtering is crucial in production, but often overlooked in practical examples.

You almost never want “*the most similar results in the entire database.*” Instead, you want results that are both similar and match certain conditions:

- Results for a specific user
- Documents tagged public
- Documents created after a certain date

And this is where things get tricky...

There are two ways to apply metadata filters:

1. Filter before search (pre-filtering)
 - The system narrows down the data using metadata filters, then runs the similarity search.
 - This approach yields more accurate results
 - But it can be slower
2. Filter after search (post-filtering)
 - System finds the most similar results and then applies metadata filters.
 - This approach is faster
 - But you might get fewer results than queried

The challenge is applying filters *before* a similarity search requires compound indexes, whereas applying them *after* can degrade result quality if the top-k similar vectors get filtered out.

So it's essential to understand how your chosen database handles metadata filtering for both query planning and debugging unexpected results.

Ranking results also have trade-offs...

Most vector databases return results sorted by distance (cosine similarity, Euclidean distance, or dot product, depending on your configuration).

But some vector databases also mix in other signals:

- How popular it is
- Custom business rules
- How recent something is

This affects what users see and how you debug “relevance issues” in production.

Know someone who wants to learn system design? Consider gifting a subscription:

[Give a gift subscription](#)

Ops Concerns: Where Theory Meets Reality

In production, failures rarely come from the algorithm itself...

HNSW and similar indexing strategies are well-proven. The operational challenges come from scaling, replication, cost management, and the

inevitable gaps between vendor documentation and reality.

Scaling Patterns: Horizontal vs Vertical

Vector databases can scale horizontally by sharding across multiple nodes or vertically by running on bigger machines with more memory and CPU.

The approach you choose depends on your dataset size, query patterns, and operational maturity...

Horizontal scaling (**sharding**) distributes your vectors across multiple nodes, with each node responsible for a subset of the data. This works well when your dataset is too large to fit on a single machine, but it introduces coordination overhead. Queries might need to hit multiple shards and merge results, which adds latency. Sharding strategies vary by database: some shard by document ID, others by vector hash, others let you define custom shard keys based on metadata.

The key operational concern is rebalancing: *what happens when you add or remove nodes, and how does the database redistribute data without downtime?*

Vertical scaling (bigger machines) is simpler.

You just run on a larger instance, but it hits limits. There's only so much memory and CPU you can throw at a single node, and past a certain size, the cost curve becomes prohibitive. Vertical scaling works well for datasets in the millions of vectors range, especially if you have spiky traffic patterns where you want to scale up temporarily rather than maintain a larger cluster permanently.

Yet a common mistake is over-engineering from the start...

Clients deploy Kubernetes clusters with auto-scaling vector database replicas for datasets that could fit on a single Postgres instance with

pgvector. You don't need Kubernetes for everything. Start with the simplest infrastructure that meets your requirements, measure actual performance under realistic load, and scale when you hit real constraints.

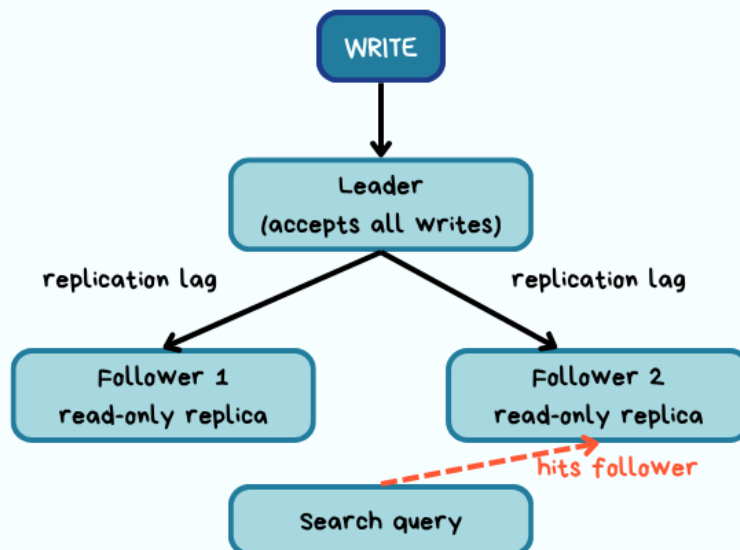
(Not because a random blog post said you need it.)

Replication for High Availability

Redundancy is great until you see the bill...then it's an upper-management meeting.

Every replica duplicates your vector storage, which, for large datasets, directly translates into cost. Most vector databases support different approaches to replication, and each offers a different trade-off among consistency, speed, and operational complexity.

Replication: Leader / Follower Lag



! The lag problem

Follower may NOT have the latest data. You write a vector, then immediately search, you might miss it.

Async replication
fast writes, eventual consistency

Sync replication
consistent, but slower writes

Leader-follower replication is the most common approach.

One server (leader) handles all writes: whenever you add or update vectors, it goes to the leader first. The leader then copies those changes to the other servers (followers), which handle read queries, such as searches. This works well for scaling search capacity and provides a backup if the leader crashes, but there is a catch: if you write new data and immediately search for it, the followers might not have received the update yet, so your search might not find it.

This delay is called *replication lag*.

Multi-leader or consensus-based replication provides stronger guarantees that all servers have the same data at all times, but comes with higher complexity and slower writes. For most AI applications, this level of consistency is overkill. If a user uploads a document and it takes a few seconds to appear in search results, that is usually acceptable.

So the decision comes down to how critical your search feature is and what your budget looks like:

- If search downtime results in lost revenue or significant user impact, run multiple replicas across different availability zones.
- If you are running an internal tool where occasional slowness is acceptable, a single server with automated backups and a clear plan for restoring quickly is probably enough.

Cost Structure: What You're Paying For

Vector databases are expensive to run because vectors are big, and computing similarity at scale requires serious resources.

So your costs break down into three key categories: *compute*, *storage*, *network*.

Compute costs come from CPU and memory needed to build indexes, serve queries, and maintain the system.

Memory is often the biggest driver. You need enough RAM to keep your index in memory for fast queries, and with millions of vectors, that adds up. (GPU acceleration can help with some operations, but adds hardware costs.)

Storage costs depend on your dataset size and replication factor.

Raw vector storage, indexes, metadata, and backups all consume disk space. This is where compression via quantization provides savings.

Reducing storage requirements by 4x directly cuts your storage costs by 4x, though you need to validate that the accuracy trade-off is acceptable for your use case.

Network costs matter if you're transferring large amounts of vector data between regions or services.

Fetching embeddings from an external API, syncing replicas across data centers, or serving high query volumes all generate network traffic that shows up in your bill.

Managed versus self-hosted is the classic trade-off...

Managed services like Pinecone, Weaviate Cloud, or Qdrant Cloud handle operations, index building, replication, backups, monitoring, and upgrades, but you're paying for that convenience and giving up some control.

Self-hosting gives you full control and can be cheaper at scale if you have the DevOps expertise (no bias here as a DevOps engineer), but you're responsible for everything: provisioning infrastructure, monitoring, upgrades, security patches, disaster recovery, and on-call when things break at 2 am.

For teams without dedicated infrastructure expertise, *managed services* often make sense. For teams with strong DevOps capabilities and cost sensitivity, *self-hosting* can be worth the operational overhead.

Monitoring and Debugging: What to Look For

Query latency is the most obvious metric to monitor, but it's far from the only thing that matters...

In production vector databases, you also need visibility into index health, resource utilization, and the quality of your search results.

Key Metrics That Matter

Query latency (p50, p95, p99) tells you how fast searches return results. Rather than looking at the average speed, track percentiles:

- p50 means half of your searches return faster than this time
- p95 means 95% of searches return faster than this time
- p99 means 99% of searches return faster than this time, showing your worst-case performance

This matters because averages hide problems.

A system with 20ms average latency but 2,000ms p99 latency means 1 in 100 users is waiting 2 full seconds for results, which is a terrible experience even if most users are fine. Also, break down latency by query type, because searches with heavy filters or requests for more results will naturally take longer than simple searches.

Index build time and memory pressure become critical when you are scaling up or recovering from a failure...

The index is the data structure that enables fast search, and rebuilding it from scratch takes time and memory. If rebuilding your index takes hours and your system crashes, that is a major outage. If building the index uses so much memory that your server runs out of memory (OOM/out of memory kill), your system will crash mid-rebuild.

So know your resource limits before you hit them.

Failed upserts and rejected queries are early warning signs that something is wrong.

An upsert is when you add or update a vector in your database. If those start failing, or if the database starts rejecting incoming search queries, it usually means you are running out of capacity, or something is misconfigured. These failures typically start small and grow before your system completely stops working, so catching them early gives you time to fix the problem before it becomes an outage.

Index freshness and replication lag tell you how up-to-date your search results are.

If a user uploads a new document and searches for it two minutes later but cannot find it, that is a real problem, even if your search speed is fast. So monitor how long it takes for new data to become searchable, and set alerts if that delay grows beyond what your users would reasonably expect.

Common Bugs You'll Encounter

Beyond standard infrastructure issues, vector databases have failure modes specific to their architecture and the AI workflows they support...

Chunking strategy mismatches occur when the chunking approach you used to create embeddings doesn't match user query patterns.

If you chunk documents into 500-token segments but users ask short, specific questions, your retrieval might return tangentially related chunks rather than direct answers.

This isn't a database bug.

It's a design mismatch that manifests as "bad search results".

Embedding model drift occurs when you change embedding models without reindexing your data. New vectors created with a different model live in a different vector space than your existing data, which

completely breaks similarity search. You need to either reindex everything (expensive, time-consuming) or maintain separate indexes for each model version (complex but enables gradual migration).

Metadata filter cardinality explosions occur when your filter predicates create too many distinct index segments or require compound indexes that don't exist. For example, filtering by `user_id` might work fine with 1,000 users but break down with 10 million users if the database tries to maintain per-user index segments.

Understanding your database's indexing strategy helps you predict where this breaks before you hit it in production.

Accuracy degradation under load is subtle but important.

Some vector databases temporarily reduce accuracy under heavy load by lowering exploration factors or skipping index layers to maintain query latency. This is often adjustable behavior, but if you're not aware it's happening, you'll see "weird search results" reported by users without obvious infrastructure issues in your metrics.

How to Debug Issues

When search results seem wrong, or queries are slower than expected, start with these steps:

- **Verify the query vector makes sense:** log the actual query embedding and spot-check that it's similar to vectors you expect to match. Embedding model issues or preprocessing differences often manifest here.
- **Check metadata filter selectivity:** if your filters reduce the candidate set to nearly zero vectors, you won't get good results. Log how many vectors remain after filtering to see if your filter is too aggressive.

- **Compare with a brute-force search on a sample:** take the same query, run it against a small random sample of your data using exact nearest neighbors, and see if the results match. This tells you whether the issue is index approximation, filtering, or something else.
 - **Review resource utilization during queries:** is CPU maxed out? Memory pressure high? Disk I/O spiking? These indicate capacity issues that affect query performance.
 - **Check index parameters and health:** when was the index last rebuilt? Are there pending updates that haven't been applied? Is replication lag high? Index staleness causes accuracy issues that look like bugs.
-

Know someone who wants to learn system design? Consider gifting a subscription:

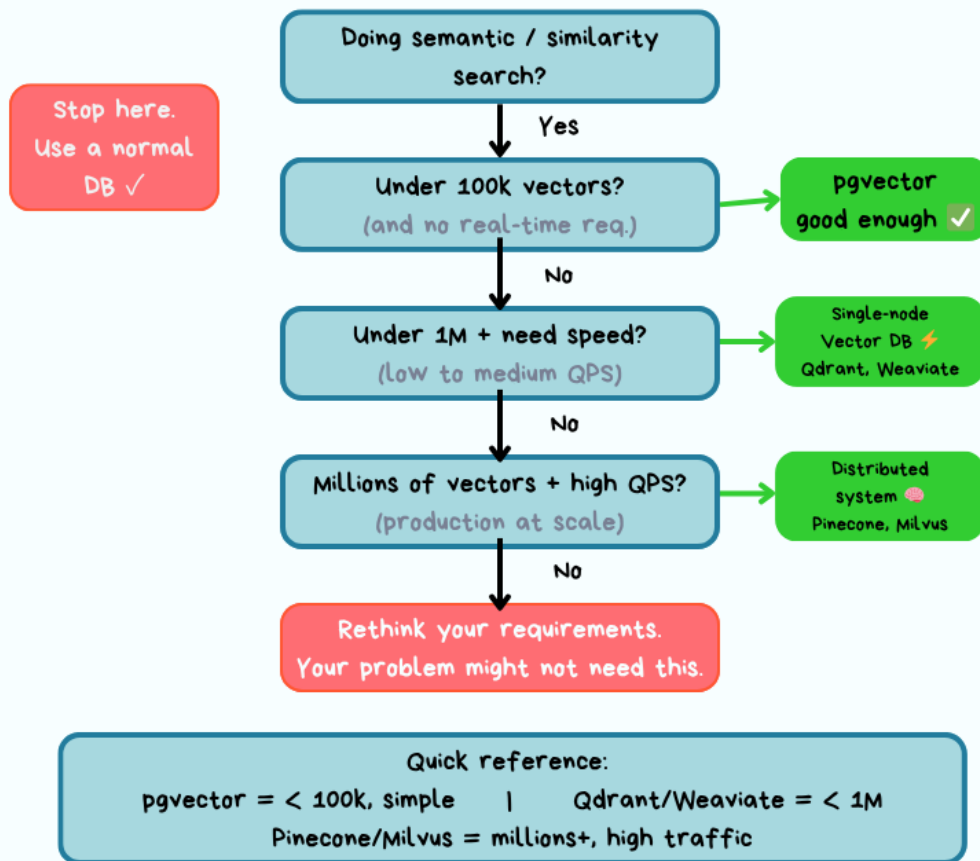
[Give a gift subscription](#)

When Do You Need a Vector Database?

Not every AI application needs a dedicated vector database infrastructure.

The decision depends on your dataset size, query volume, latency requirements, and tolerance for operational complexity.

Do You Need a Vector DB?



Under 100k Vectors + Moderate Query Volume

Use: *pgvector* in PostgreSQL (or SQLite with *sqlite-vss* for even smaller datasets)

Your data is small enough to run on a single database, and your search volume is not high enough to overwhelm a general-purpose database. If you are already running PostgreSQL, your team knows how to operate it, and you have backups and monitoring in place, there is no reason to add another system just for vector search.

Trade-offs: Search will not be as fast as a dedicated vector database, and you will hit limits as your data grows. But for many applications, “good enough” performance with much simpler infrastructure is the right call.

Under 1M Vectors + Need Sub-50ms Latency

Use: Dedicated vector database (Qdrant, Weaviate, Milvus, or managed options)

Sub-50ms latency means your search results need to come back in under 50 milliseconds, fast enough that users do not notice any delay. At this scale, pgvector starts struggling to meet that bar, and a dedicated vector database is worth the extra complexity.

Trade-offs: You are now running an additional database, which means more to monitor, back up, and maintain. Costs go up because you are paying for dedicated infrastructure. But search performance improves significantly, and you get access to features such as **hybrid search** (combining keyword- and meaning-based search) and enhanced filtering options.

Multi-Million Vectors + High QPS + Multi-Tenant

Must have: Distributed vector database with horizontal scaling

At this scale, a single database server cannot handle the load. You need your data split across multiple servers (sharding), with copies for backup (replication), and a system that automatically routes each search query to the right server.

Multi-tenant means multiple customers or teams using the same system, with their data kept separate.

Trade-offs: This is the most complex setup. Debugging problems across multiple servers is hard, it's expensive, and you need engineers who

understand distributed systems. But if you are serving millions of searches per day against tens of millions of vectors, there is no simpler alternative.

Operational Overhead Factor

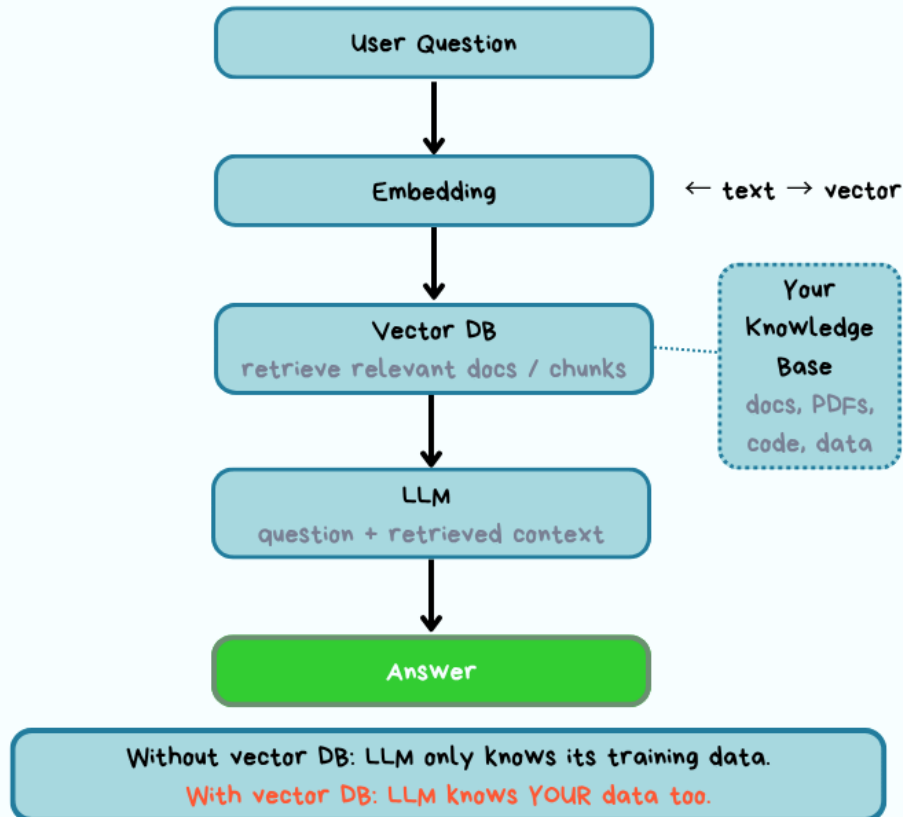
Before deciding which option to use, think *honestly* about your team's capacity.

Every additional database you run means more things to monitor, back up, secure, upgrade, and fix when something breaks at 2 am. If your team is already stretched thin, adding a dedicated vector database for a feature that could run on pgvector creates **unnecessary risk**.

On the other hand, if you have engineers who specialize in database operations and search is the core feature of your product, investing in dedicated infrastructure makes sense.

Vector Databases in AI Infrastructure Landscape

RAG: Where Vector DB Fits



Vector databases solve one problem exceptionally well: *finding similar items in high-dimensional space without checking everything.*

They do this by trading perfect accuracy for massive speed gains through specialized indexes like HNSW. But here's what matters for your next architecture decision: *most teams don't need a dedicated vector database right away.*

- If you're under 100k vectors, PostgreSQL's pgvector is probably enough.

- If you're building a customer-facing feature that needs sub-50ms search, a dedicated vector database justifies the operational overhead.
- If you're serving millions of searches per day across tens of millions of vectors, you need distributed infrastructure, and there's no way around it.

The mistake isn't choosing the wrong database.

The mistake is adding infrastructure complexity before you need it, or waiting until your system is on fire before you add the infrastructure you should have deployed months ago.

Start simple. Measure religiously. Scale intentionally.

Your future on-call self will thank you.

TLDR

Vector databases enable fast semantic search by:

- Converting text into high-dimensional coordinates (embeddings)
- Using specialized indexes (HNSW) to navigate through those coordinates instead of checking everything
- Trading perfect accuracy for 100x+ speed improvements

When you need one:

- Under 100k vectors → Use pgvector in PostgreSQL
- 100k-1M vectors + need <50ms latency → Dedicated vector DB (Qdrant, Weaviate, Milvus)
- Multi-million vectors + high traffic → Distributed vector DB

Key operational concerns:

- Quantization reduces storage costs 4x with <1% accuracy loss
 - p99 latency matters more than averages (1% of users experiencing 2-second delays is a problem)
 - Every replica = doubled costs
 - Start simple, measure actual performance, scale when you hit real limits
-

👋 I'd like to thank **Maxine** for writing this newsletter!

Also, find her on:

- [Substack - I Love DevOps](#)
- [LinkedIn](#)
- [Threads](#)

She's a cloud infrastructure engineer.

When she's not fighting with Terraform or explaining why "it worked on my machine" isn't a valid deployment strategy, she's probably trying to convince her son that vegetables are food.

Know someone who wants to learn system design? Consider gifting a subscription:

[Give a gift subscription](#)



👋 Find me on [LinkedIn](#) | [Twitter](#) | [Threads](#) | [Instagram](#)

Thank you for supporting this newsletter.

You are now 210,001+ readers strong, very close to 210k. Let's try to get 211k readers by 27 April. Consider sharing this post with your friends and get rewards.

Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share

Further Reading

- [pgvector GitHub](#) - PostgreSQL extension for vector search
- [Weaviate Documentation](#) - Open-source vector database
- [Milvus Documentation](#) - Scalable open-source option
- [Pinecone Documentation](#) - Managed service